# Monolithic Architecture

Software Architecture Notes — Knowledge Base

---

## 1. What is it?

A monolithic architecture packages an entire application — UI, business logic, and data access layer — as a single deployable unit. All modules run in the same process and typically share a single database.

### 2. Core Characteristics

- Single codebase and single build/deploy artifact (e.g., one WAR/JAR, one container image).
- Modules communicate via in-process function/method calls — fast, no network overhead.
- Usually organized in layers: presentation, business logic, data access.
- Single shared database is common.

### 3. Advantages

- Simple to develop, test, and deploy early on — one thing to build and run.
- Easier to reason about transactions (single ACID transaction across the whole app).
- Lower operational complexity — no service discovery, no distributed tracing needed.
- Simpler local development setup.

### 4. Disadvantages

- Scaling means scaling the entire application, even if only one module is under load.
- Large codebases become harder to understand and onboard onto over time.
- A bug or memory leak in one module can bring down the entire application.
- Slower deployments as the codebase grows — every change redeploys everything.
- Technology lock-in — the whole app usually shares one language/framework.

### 5. The 'Modular Monolith'

A modular monolith keeps a single deployable unit but enforces strict internal module boundaries (clear interfaces, no direct cross-module DB access). This gives many of the maintainability benefits associated with microservices while avoiding distributed-systems complexity, and is often a good middle ground.

### 6. When to Use

Ideal for startups, MVPs, and small-to-medium teams where development speed and simplicity matter more than independent scalability. Many successful large systems (including some that later moved to microservices) started as monoliths.

## Monolith vs Microservices — Quick Comparison

| Aspect | Monolithic | Microservices |
|---|---|---|
| Deployment | Single unit | Independent per service |
| Scaling | Whole app | Per service |
| Complexity | Low (early), high (later) | High from the start |
| Team structure | Works for small teams | Suits multiple autonomous teams |
| Data | Usually shared DB | Database per service |